

The Good, the Bad, and the Ugly:

The WID.world Stata Guide of Rules and Best Practices

Thomas Blanchet

January 15, 2020

Contents

1	Project Structure	2
1.1	Stata's Project Management System	2
1.2	Master Files	3
1.3	Directories of a Project	5
2	Code Style	6
2.1	Naming Variables	6
2.2	Spacing	7
2.3	Blocks of Code and Code Alignment	9
2.4	Code Indentation	10
2.5	Comments	11
2.6	Line Width	12
3	Tricks, Common Errors, and Best Practices	14
3.1	Never Modify a Data File Multiple Times	14
3.2	Use the <code>assert</code> Function	15
3.3	Debugging: <code>set trace on</code>	16
3.4	Testing Equality	16
3.5	Testing Missing Values	17
3.6	Comparing Numbers	17
3.7	Using <code>foreach</code> Loops	18
3.8	Controlling Loops	18
3.9	Avoid Creating Unnecessary Permanent Files	18
3.10	Using the <code>merge</code> Function	19
3.11	Rectangularize Panel Data	19
3.12	Exporting Graphs	19

Chapter 1

Project Structure

1.1 Stata's Project Management System

For any sizable project, use Stata's project management system. It will let regroup or organize all your Stata files into a single practical interface. To create a Stata project, go to **File > New > Project**. It will create a `.stpr` file. You can open this file with Stata's do-file editor: a sidebar will appear on the left of the window. Whenever you create a new do-file, do it from this sidebar, so that it will appear there. After adding (or removing) a file, do not forget to save the project using **File > Save project**.

File Naming Conventions Use lowercase names for all your files. Never use uppercase letters, as this will create problems. Indeed, not all file systems are case-sensitive (Windows's file system is case-sensitive, but macOS's usually isn't). Therefore, if you get the case wrong when working with files in your code, it may end up working on some platforms but not others. For this reason, it is better to stick with fully lowercase or fully uppercase file names, and lowercase looks better and more natural. Follow this rule not only for your do-files, but also for any file that your code creates (data, graphs, tables, etc.)

Separate words in file names using hyphens: e.g. `import-macro-data.do`. Always use the `.do` extension for do-files (and `.ado` for ado-files, if any).

File names should start with a verb and indicate what the do-file *does*, preferably using complete words. For example:

- use `import-macro-data.do` rather than `macro-data.do`

- use `impute-missing-values.do` rather than `imputations.do`
- use `run-regressions.do` rather than `reg.do`
- use `calculate-descriptive-statistics.do` rather than `stat-desc.do`

File Grouping and Organization Stata’s project management system lets you group your various files in virtual folders. Take advantage of this functionality to regroup your files by function.

The do-files are generally meant to be executed in a given order. Make that order apparent in the way you name your files and your groups. Stata will show them in alphabetical order, so adopt the following convention:

- 01-import-data
 - 1a-import-macro-data.do
 - 1a-import-micro-data.do
 - 1b-combine-macro-micro.do
- 02-run-analysis
 - 2a-calculate-descriptive-statistics.do
 - 2b-run-regression.do
- ...

Note the use of two digits to indicate the order of groups: this will make the groups appear in proper order even if you have more than nine of them.¹ The files are named with a digit that references the group they belong to, followed by a letter indicating the proper sequential order within the group. If two files can be executed in any order, you can use the same letter (e.g. `1a-import-macro-data.do` and `1a-import-micro-data.do`).

1.2 Master Files

Any project should have at least two “master” do-files, and generally three. Put these master do-files in a group named `00-master` so that they appear first in the project sidebar. Their role is to make sure that the Stata environment is properly set up, and to call to other do-files in the proper order.

“Install” Master File The first master do-file — if required — should be called `0a-install.do`. It should contain the command `ssc install pkgname` for every user-written package that the rest of the code requires. This way, anyone that wants

¹You can use three digits if you expect more than 99 groups, but that is unlikely.

to run the code just has to execute this do-file once to ensure that they have all the required packages installed.

“Setup” Master File The second master do-file should be called `0b-setup.do`.² It should define all the required paths as global macros for the program to run properly. You will need at least two macros: one called `$do` that contains the do-files, and one called `$work` that contains intermediary data files. But you may want to define additional paths for raw data, etc.

Use a simple `if` condition to determine which of the main contributors to the project is currently executing the code, and set the paths accordingly to that the program doesn’t need to be changed for every user. For macOS and other Unix systems, take advantage of “~” that works as a shortcut to the home directory. So you would typically have:

```
// Windows users
if substr("`c(pwd)'", 1, 16) == "C:\Users\JaneDoe" {
    global root "C:\Users\JaneDoe\Projects\MyProject"
}
if substr("`c(pwd)'", 1, 16) == "C:\Users\JohnDoe" {
    global root "C:\Users\JohnDoe\Projects\MyProject"
}
// macOS and other Unix users
if substr("`c(pwd)'", 1, 1) == "/" {
    global root "~/Projects/MyProject"
}
```

“Run” Master File The third master do-file should be called `0c-run.do`. It should call all the other do-files, in the correct order, using:

```
do "$do/1a-do-something.do"
do "$do/1b-do-something-else.do"
```

Put various comments in this file to explain what each do-file does and the general structure of the project.

²Or `0a-setup.do` if there is no `0a-install.do` — you get the idea.

1.3 Directories of a Project

Put the various components of your project (data, code, etc.) in separate folders. But try to limit the number of nested folders: more than two levels should remain quite rare, otherwise your project quickly become a labyrinth. We recommend using the following structure:

- code/
 - 0a-install.do
 - ...
- work-data/
 - data.dta
 - ...
- raw-data/
 - data-source1/
 - data-file.xlsx
 - ...
- figures/
 - figure1.pdf
 - ...

The “code” Folder This folder contains all of the do-files, ado-files and any other piece of code (including bits written in a different language, such as R). Note that this folder does not need to reproduce the structure of the Stata project (see section 1.1). In fact, it is better to keep all the files as-is, with a flat structure, and manage their organization through the Stata project sidebar only, otherwise you will have to maintain two pieces of arborescence at once.

The “work-data” Folder This folder contains the data files that are produced by your code and used by several of your do-files. These files should be saved in .dta format.

The “raw-data” Folder This folder contains all the source files that are used by your programs. You can divide it into one subfolder for each source, each with a README.md file that indicates where the data comes from.

The “figures” and Other Output Folders These folders contain the various formatted outputs of your programs, such as figures and tables. You can create several of those if you have many different outputs.

Chapter 2

Code Style

2.1 Naming Variables

Naming variables is notoriously difficult. The names must strike an adequate balance of expressiveness, precision, and length. The length factor is especially important in Stata given that variable names are limited to 32 characters. If you have variable names that are near this limit, it can create problems down the road when using `reshape wide` because the command will try — and fail — to create longer names.

Getting the Names Right Abbreviations are fine if they are sufficiently explicit or common (e.g. `coef` for `coefficient`, `sd` for `standard_deviation`), but should not go too far. When a new variable is a small modification of an existing variable, you will often want to indicate it by having a name for the new variable that derive from the one of the existing variable. When doing so, beware of lazy suffixes such as “2”, and prefer more descriptive ones. For example, instead of `gen var2 = string(var)`, you should use `gen var_str = string(var)`.

You can make exception for temporary variables in tiny pieces of code. For example, variables such as `var2` can be used in some manipulations if they are rapidly dropped. They should not be used for important data, otherwise those will end up named `var2` or `var3` or `var4` without any clear reference to the original variable they derive from. Very short, non-descriptive names (typically one letter) should also be limited to small bits of code.

Groups of variables that are related should start with the same prefix, so that you can quickly create a varlist out of them (e.g. `income_labor income_capital` can be abbreviated as `income_*`).

Typographic rules Stata variables are always in lowercase. When you import data from an external source that uses some uppercase letters, their names should progressively be replaced by lowercase ones. If a variable name is made up of several words, separate them using an underscore. So write:

```
generate my_sweet_variable = 0
```

The following, on the other hand, are less readable and therefore discouraged:

```
generate mysweetvariable = 0
generate mySweetVariable = 0
generate MySweetVariable = 0
generate MYSWEETVARIABLE = 0
```

2.2 Spacing

Proper spacing is an important part of readability. Beginners have a tendency not to put spaces between symbols unless required by the language to save on keystrokes. As a result, their code ends up very dense, compact and hard to read. Going too far in the other direction — putting a lot of spaces everywhere — also defeats the purpose. Spacing is there to emphasize the syntactic meaning of the code, so it should be used, but not indiscriminately. To achieve the proper balance, follow these simple rules.

General rules Always put spaces around the assignment operator (i.e. the equal sign `=`) and comparison operators (i.e. `==`, `<=`, `>=`, `<`, `>`). Also put spaces around “and” (`&`) and “or” (`|`). Do not put additional spaces around parentheses or brackets. Put a space after a comma (`,`) or colon (`:`), but not before (as you would in plain English). Therefore, you should write:

```
by z: gen a = b[_N] if (c <= d) & inlist(x, 1, 2)
```

and not:

```
by z:gen a=b[ _N ] if(c<=d)&inlist(x,1,2)
```

There are some exceptions for which it is better not to put spaces around the equal sign. The main one concerns lists of expressions, like the one you can find in the `collapse` command. That is, you can write:


```
collapse (sum) a=b x=y, by(group)
```

which does a better job of separating `a=b` from `x=y` than:

```
collapse (sum) a = b x = y, by(group)
```

Mathematical expressions In mathematical formulas, spacing should reflect the priority of the various operations. For example, multiplication has the priority over addition. So if you mean $a \times b + c \times d$, you should write:

```
display a*b + d*c
```

which is better than the other alternatives:

```
display a*b+d*c      // Less readable
display a * b+d * c  // Even worse: looks like a*(b + d)*c
```

Depending on the complexity of your expression, you may even want to put redundant parentheses to improve clarity:

```
display (a*b) + (d*c)
```

For consistency, keep following these conventions even when not mixing additions and multiplications:

```
display a*b           // Not: a * b
display a + b         // Not: a+b
display (a + b)*(d + c) // Not: (a+b)*(d+c)
```

You can put spaces around multiplications when using the exponent symbol `^` to indicate its highest priority:

```
display a^b * c^d
```

Put spaces around the minus sign (`-`) when it is an operation, but not to change the sign of a number:

```
display -a + b // Not: - a + b
display b - a // Not: b-a
```

2.3 Blocks of Code and Code Alignment

Group the lines of your code in logical blocks: when several lines of code are part of the same task, they should belong to the same block. This will make it easier to identify the different parts of your code in the future. These blocks should be separated by a single empty line, and preceded by a comment that indicates what the corresponding task does.

Within a block, if you have several similar lines of codes in a row, try to align key operators (such as the `=` sign) across the lines. This will make it a lot easier to spot inconsistencies and therefore identify mistakes. The common situation is when you have several assignments:

```
generate one_variable      = some_data + another_data
generate another_variable = some_data + yet_another_data
```

Notice how the definition of the variable is clearer than if we had not aligned the `=` signs:

```
generate one_variable = some_data + another_data
generate another_variable = some_data + yet_another_data
```

In particular, we can identify spelling mistakes more quickly in the aligned version:

```
generate one_variable      = some_data + another_data
generate another_variable = some_dta + yet_another_data
```

than in the non-aligned one:

```
generate one_variable = some_data + another_data
generate another_variable = some_dta + yet_another_data
```

2.4 Code Indentation

Always indent blocks of code that are inside brackets, i.e. blocks of code that follow a `if`, `while`, `foreach` or `forvalues` statement. This is a nonnegotiable rule of good programming in every language.¹ Unfortunately that principle is not always taught to people learning Stata, so that a lot of Stata code out there fails to adhere to it. As your Stata code becomes more complex (with several layers of imbricated blocks) it becomes quickly unreadable. There is never a good reason not to follow this rule. Get used to it right now — it should become a reflex.

How to indent is subject to debate. Some people use spaces, other use tabs. Using spaces is arguably cleaner and ensures that the code will look the same on every platform, whereas tabs can be interpreted differently by different text editors. But typing several is also more of a hassle than typing a tab. Many modern editors can convert tabs to spaces directly, but unfortunately this isn't the case of Stata's do-file editor.

Whether you choose to use tabs or spaces, make sure that the code editors will display them consistently. By default, Stata's do-file editor displays tabs as four spaces, so if you use spaces to indent your code, four spaces is a good choice. Some people prefer to use two spaces or eight spaces instead: four spaces is a better trade-off between readability and line length.

With indentation, your code should look like this:

```
forvalues i = 1/10 {  
    foreach v of varlist var1-var10 {  
        if (mod(`i', 3) == 0) {  
            replace `v' = `v' - foo`i'  
        }  
    }  
    if (`i' > 1) {  
        append using "data"  
    }  
    save "data", replace  
}
```

which is equivalent to, yet a lot more readable than this:

¹So much so that certain languages (like Python) have made it part of their syntax.

```

forvalues i = 1/10 {
  foreach v of varlist var1-var10 {
    if (mod(`i', 3) == 0) replace `v' = `v' - foo`i'
  }
  if (`i' > 1) append using "data"
  save "data", replace
}

```

You should actually use indentation for every command that delimits a block of code, even if it doesn't explicitly rely on brackets, for example `program define`:

```

program define my_program
    // Put indented code here
end

```

You can also indent code between the commands `preserve` and `restore`. This is a good idea if the code between the two commands is short. This may be less justifiable if the code is longer: use your best judgment.

2.5 Comments

To write comments in your code, use `// Comment` rather than `/* Comment */` or `* Comment`.

Using `/* ... */` or `* ...` is useful to comment out bits of code during debugging. But `// ...` is better for permanent ones. The syntax `/* ... */` is less practical than `// ...` for one-line comments. It more practical for putting blocks of texts in comments, but is also worse at making blocks of commenting text stand out. As for `* ...`, it is an odd Stataism, while `// ...` is more common across programming languages — and thus more readable to a wide audience. Using `* ...` is also not as readable due to its similarity with the multiplication operator.

Use comments to explain what you are doing, but also to delimit the various parts of your code. Make the main titles for various parts of your code stand out using the following style:

```

// ----- //
// Your title here
// ----- //

```

The `// --- //` should take the full line width, i.e. 80 characters (see section 2.6).

2.6 Line Width

With today's wide computer screens, it has become possible to fit 200 or 300 characters into a single line of text without having to scroll. Please don't.

Extremely long lines of code are harder to work with. People will one day look at your code on smaller devices than the 32-inch screen that you may have at work: small laptops, tablets, or even smartphones. You're making it harder for them to read your code if you use lines of code that only fit on a big screen. Many people also take advantage of large screens to put several windows side-by-side, say to look simultaneously at the code, the data and the console. They will suffer from very long lines of code.

Historically, computer screens were only able to fit 80 characters. This led to the usual recommendation that lines of code should not exceed this limit. With modern equipment, this advice may sound dated, but it is still a good reference. You can make exceptions to that rule, but keep it as a point of reference. If a line of code starts to exceed this length, start to think of ways to avoid this. Use your best judgment: you may ultimately decide to keep the line as is if it only exceeds the limit by a few characters and breaking it would be too much of a hassle. But at least think about it. To easily spot lines of code that exceed 80 characters, display the page guide in Stata's do-file editor: go to **Preferences > Windows > Do-file editor** and check "Page guide" with "at column" set to 80.

There are two solutions to break a long line of code. The first one is to rewrite it as two commands. For example, long math expressions can be broken as such:

```
// Instead of:
generate foo = (first long expression)*(second long expression)
// Write:
generate bar1 = first long expression
generate bar2 = second long expression
generate foo = bar1*bar2
```

The second solution is to use `///`, which has both the effect of starting a comment and indicating to Stata's interpreter that the command continues to the next line. Make sure to indent the parts of the command that come after `///`. To further

improve readability, try to break long lines in logical ways, if possible by commenting things along the way. When breaking up math expression, have new line start by an operator rather than end by it. For example:

```
// Instead of:
generate wealth = stocks + bonds + housing + ///
business_wealth + pension - ///
mortgages - other_debt
// Write:
generate wealth = stocks + bonds + pension    /// Financial wealth
                + business_wealth + housing /// Real wealth
                - mortgages - other_debt      // Debt
```

Chapter 3

Tricks, Common Errors, and Best Practices

3.1 Never Modify a Data File Multiple Times

As projects become larger, they will be several data files in your `work` directory that your code will be using. In order to get reproducible results, you should always make sure that a given data file is created/modified by a single do-file. Subsequent do-files can access and read that file as needed, but should never modify it.

If you don't follow this rule, your code will not be easily reproducible. Results may change from one execution to the next. You will execute do-files not knowing what version of the data file they will use. It will create problems.

Take the following example. The file `do-first.do` creates the file `data.dta` with a variable `var` equal to `1`. The file `do-second.do` loads `data.dta` and multiplies `var` by two, and saves it to the same file. The file `do-third.do` report the value of `var`. Every time you execute `do-second.do` and `do-third.do`, you will get a different result. But if `do-second.do` had saved its result to a different file, there would be no problem.

The example may sound artificial, but when you work on big projects you will be tempted to do analogous things. Do not give in to that temptation as it will create problems.

3.2 Use the `assert` Function

The `assert` function allows you to check that something about your data is true. Put such checks in your code regularly to make sure that everything is going as expected. While it may look unnecessary now, it will save a lot of time down the road.

In general, as you write your code, you will do many checks “on the fly” to make sure that your code is working as expected, and that the prerequisites for doing certain data manipulation are satisfied. This is good and important, but insufficient. Indeed, you will be executing that code many times, with slightly different data. While your code might be working now, it might break in the future, because some condition is not satisfied anymore. Assuming that your code fails silently, you will waste a lot of time figuring out where it goes wrong — or worse, you may not even notice that it has gone wrong.

The problem isn’t that you haven’t checked your code, the problem is that you haven’t done so in a systematic and permanent way. Using `assert` solves that problem. It will make your code stop unless some condition is satisfied. Every time you find yourself checking something about your data, or checking that your code has worked properly, you should write it as an `assert` command and leave it permanently in the code, with a comment indicating what the test means. If the code stops working at some point in the future, it will immediately warn you.

You can use `assert` to check something about all of the data:

```
// Check that the quantile function 'q'  
// is increasing with the rank 'p'  
sort p  
assert (q[_n + 1] > q) if _n < _N
```

or to check a single value:

```
// Check that the bracket shares sum to one  
summarize bracket_share  
assert r(sum) == 1
```

When using the `merge` command, you can directly use the option `assert(...)` to make sure that the merge is happening as expected.


```
// Make sure that the two datasets match perfectly
merge 1:1 id using "data.dta", assert(match)
```

3.3 Debugging: `set trace on`

When debugging, especially if using macros, you will want to use `set trace on`. When you type `set trace on` in the console, Stata will enter into a debugging mode. It will show you line by line how it executes the code, in particular replacing macros by their value, and expending variable lists. This is extremely helpful to understand what Stata is doing at every stage of your code. When you are done debugging, use `set trace off` to go back to the normal mode.

By default, Stata will display every instruction that it is executing, including the ones located inside internal Stata commands. This output can become overwhelming. You can use `set tracedepth 1` to only show how Stata is interpreting your own code.

3.4 Testing Equality

When you test the equality of two variables, beware of the type of these variables. There is no problem when testing the equality of two strings or two integers, but testing the equality of two real numbers can fail in mysterious ways. For example:

```
assert 1.1 + 2.2 == 3.3 // Yields an error
```

will yield an error, but:

```
assert 1.2 + 2.1 == 3.3 // Does not yield an error
```

will not.¹ Yet both expressions look mathematically correct — and very similar. This is because computers have no way store real numbers exactly, so they use approximate representations of real numbers. You should therefore avoid comparing real numbers directly. This also means that you should not merge two datasets based on an identifier that is a real number (strings are the safe option for identifiers). You should also not group observations using `by:` based on a real variable. Beware of this when using integers, because Stata has a tendency to turn large integers

¹Results may be different on your own machine, but you get the point.

into real numbers automatically. If you really have to test the equality of two real numbers, use the relative difference function with a nonzero tolerance:

```
assert reldif(1.1 + 2.2, 3.3) < 1e-6 // No error
```

3.5 Testing Missing Values

There are several missing values concepts in Stata. The main one, and also the most common, is the “system missing value” (`.`). But there is also a set of “extended missing values” (`.a`, `.b`, ..., `.z`). Importantly, Stata considers that `.` is different from `.a`. Therefore, to check if a value is missing, you should use the function `missing()`:

```
replace foo = 0 if missing(bar)
```

which is safer and more explicit than typing:

```
replace foo = 0 if (bar == .)
```

3.6 Comparing Numbers

Beware of missing values when comparing numbers. Indeed, Stata considers that:

$$\text{all nonmissing numbers} < . < .a < .b < \dots < .z$$

so that `1000 < .`, for example, is true. When your data contains missing values, make sure that those are being handled properly. A typical error is to create a dummy variable as:

```
generate dummy = (foo > 0)
```

which will lead to `dummy` equal to one when `foo` is missing. Instead, you should do:

```
generate dummy = .  
replace dummy = 1 if !missing(foo) & (foo > 0)  
replace dummy = 0 if !missing(foo) & (foo <= 0)
```

3.7 Using `foreach` Loops

Stata has two syntax for `foreach` loops: `foreach ... of ...` and the more permissive `foreach ... in ...`. Try to use `foreach ... of ...` unless you have to use the other version. Indeed, `foreach ... of ...` is safer because it will check the arguments in advance. It can also be more practical for looping over variables because it lets you use the power and flexibility of varlist expansions.

3.8 Controlling Loops

For some complex operations, you may want to skip an iteration or exit a loop prematurely if a condition is satisfied. To that end, have a look at the function `continue` (to skip an iteration) and `continue, break` (to exit a loop). These commands can often help you avoid hard-to-read loop constructs using auxiliary variables.

3.9 Avoid Creating Unnecessary Permanent Files

If a data file is not meant to be used by several do-files, then it should not be taking unnecessary space in the `work-data` directory (see section).

To avoid doing this, you should the command `tempfile`. It will store the name of a temporary file inside a macro that you can then use for quick data manipulations. The file will automatically be deleted at the end of the execution.

The typical use of `tempfile` is to combine several files:

```
tempfile combined_data
forvalues year = 2000/2020 {
    use "data`year'.dta", clear
    if (`year' > 2000) {
        append using "`combined_data'"
    }
    save "`combined_data'", replace
}
// Use the combined data here
```

3.10 Using the `merge` Function

You probably know the `merge` function, but here are a few tips for using it well. First, two things that have been said elsewhere but are worth repeating here. Never perform a merge using a real number as your identifier. To identify observations, use integers or string only. Second, whenever possible, use the option `assert(...)` to make sure the merge is happening as expected.

By default, `merge` create the variable `_generate` to indicate if observations are present in the master dataset, the using dataset, or both. This is useful for debugging, but once this done you should use the option `nogenerate` to avoid creating the unnecessary variable. If you need to only keep certain observations, use the option `keep(...)` rather than relying on `_generate`. If you only need a few variables from you using dataset, use the option `keepusing(...)`.

You should never use `merge n:n`. It is often labeled “risky” or “dangerous” in various documentations. This is not quite correct. The option `n:n` is not risky or dangerous. It’s madness. It should never have been included as an option. You must never use it. If you think you have found a use case for `merge n:n`, you’re wrong. It usually means that you haven’t identified your observations well enough in your datasets. You should modify them and do so in an explicit way. Using `merge n:n` means that Stata will do this for you in indecipherable and non-reproducible ways.

If you ever find yourself wanting to do something that feels like a `merge`, but that `merge` cannot do, have a look at `joinby`. This is `merge`’s lesser known cousin. It does something different from `merge`, so don’t think of them as equivalents. But it can be useful for some purposes.

3.11 Rectangularize Panel Data

If you find yourself wanting to “rectangularize” panel data, know that there is a Stata function to do just that: `tsfill`. It fills gaps in panel data with new observations, which contain missing values.

3.12 Exporting Graphs

It is preferable to export graphs in as PDF files. They will look much better in final documents, especially when printed. Some figures (especially scatter plots) may create problems because they are too heavy for this. In that case you can consider

the PNG or TIFF formats, but make sure that the resolution is enough. Medium resolution may look fine on a screen but will look terrible on paper.